

مستندات فنی جامع

پروژه نهایی درس طراحی کامپایلر

COMPOILER

معماری ماژول‌ها، انتخاب الگوریتم‌ها، گرامر EBNF، محدودیت‌ها و دستورالعمل کامل تست

طراحی کامپایلر	نام درس
بهار ۱۴۰۴-۱۴۰۵	نیم‌سال
دکتر علانیان	استاد
علیرضا قربانی	سرپرست آموزشی
آریا فتحعلیان محمد مهدی توحیدی نسب	نام دانشجو / اعضای گروه
40225793 40216093	شماره دانشجویی

تاریخ تهیه: مرداد ۱۴۰۵

هدف اصلی — این سند دقیقاً برای پوشش الزام تحویل پروژه تهیه شده است: معماری ماژول‌ها، انتخاب و توجیه الگوریتم‌ها، محدودیت‌های شناخته‌شده و دستورالعمل مرحله‌به‌مرحله اجرای تست‌ها. همچنین گرامر EBNF، راهنمای CLI، روش اجرای Docker و چکلیست نهایی تحویل در آن آمده است.

پروژه COMPOILER یک Front-End ماژولار برای زیرمجموعه‌ای آموزشی از زبان C است. سیستم کد منبع را دریافت می‌کند، توکن‌سازی و تجزیه نحوی را انجام می‌دهد، AST می‌سازد، تحلیل معنایی و نوع را اجرا می‌کند، گراف جریان کنترل و گراف فراخوانی تولید می‌کند و ابزارهای IDE مانند Completion، Hover، Go-to-Definition، Find References و Safe Rename را در اختیار کاربر می‌گذارد.

دامنه مستند

- فاز اول: Lexer، Parser، AST، بازیابی خطا و Syntax Highlighter.
- زیرساخت مشترک: Diagnostic، Symbol، Reference و Scope Tree.
- فاز دوم: تحلیل معنایی، Type Checking، Completion و Hover.
- فاز سوم: CFG، تحلیل جریان داده، Call Graph، Navigation، Refactoring و Dead Code.
- بخش‌های تکمیلی و امتیازی: Code Metrics، نمایش گرافیکی CFG و Call Graph و Docker Deployment.
- راهنمای اجرا، دستورات CLI، تست ورودی‌های صحیح و خطا دار و چکلیست تحویل.

وضعیت فعلی قابل استناد

در اجرای مشاهده‌شده، Image داکر با نام compiler ساخته شد، برنامه با docker run اجرا شد، خروجی فازهای مختلف نمایش داده شد و Prompt تعاملی <compoiler ظاهر شد. این موضوع اجرای موفق برنامه در Container را نشان می‌دهد. نتیجه تست‌های واحد باید هنگام تحویل روی نسخه نهایی دوباره اجرا و ثبت شود.

- اجرای اصلی پروژه: python3 mainkon.py یا python mainkon.py.
- رابط کاربری: CLI تعاملی با Prompt برابر <compoiler.
- خروجی‌های قابل تولید: highlighted.html، cfg_report.html و callgraph_report.html.
- Docker: فایل Dockerfile موجود است و اجرای تک‌فرمانی با docker run انجام شده است.
- پیش از تحویل، نام دانشجو، شماره دانشجویی و لینک Repository در صفحه اول و README تکمیل شود.

فهرست مطالب

۱ معرفی پروژه و معماری کلان

۲ ساختار پوشه‌ها و مسئولیت ماژول‌ها

۳ فاز اول: Lexer و قواعد رسمی Tokenها

۴ Parser، گرامر AST، EBNF و Panic Mode

۵ Syntax Highlighter و زیرساخت Diagnostic/Symbol/Scope

۶ فاز دوم: Semantic Analysis و Type Checking

۷ Hover و IntelliSense: Completion

۸ فاز سوم: CFG و Data-Flow Analysis

۹ Dead Code و Call Graph، Navigation، Safe Rename

۱۰ بخش‌های تکمیلی: Metrics و گزارش‌های گرافیکی

۱۱ رابط CLI و راهنمای استفاده

۱۲ Docker Deployment

۱۳ راهبرد تست و سناریوهای ارزیابی

۱۴ محدودیت‌های شناخته‌شده

۱۵ چک‌لیست تحویل و دفاع

پیوست‌ها کد نمونه، خروجی مورد انتظار

نکته درباره شماره خط و ستون — موقعیت‌های Cursor در ابزارهای Completion، Hover، Navigation و Rename از نوع یک‌مبنا هستند؛ یعنی اولین خط و اولین ستون با عدد ۱ مشخص می‌شوند. هر خط خالی داخل رشته سه‌گانه Python نیز در شماره خط محاسبه می‌شود.

۱. معرفی پروژه و معماری کلان

هدف پروژه ساخت یک خط لوله کامل Front-End کامپایلر همراه با قابلیت‌های تحلیل کد و ابزارهای شبیه IDE است. زبان هدف، زیرمجموعه‌ای از C است و تمرکز اصلی روی مفاهیم طراحی کامپایلر، تحلیل ایستا و معماری ماژولار قرار دارد؛ نه پیاده‌سازی کامل استاندارد ISO C یا تولید کد ماشین.

۱.۱ جریان داده اصلی



جریان اجرا از متن خام شروع می‌شود. Lexer متن را به Token تبدیل می‌کند. Parser از Token‌ها AST می‌سازد. فاز دوم AST را با Symbol، Scope و Type غنی می‌کند. فاز سوم CFG، Data-Flow و Call Graph را می‌سازد و ابزارهای Navigation، Rename و تشخیص Dead Code را ارائه می‌دهد. در پایان، خروجی‌ها از طریق CLI و Terminal، HTML در دسترس هستند.

۱.۲ اصول طراحی

- جداسازی مسئولیت‌ها: هر الگوریتم اصلی در فایل مستقل پیاده‌سازی شده و mainkon.py فقط هماهنگ‌کننده است.
- حفظ فاز اول: ماژول‌های جدید از طریق import به برنامه اضافه شده‌اند و منطق فاز اول به صورت مستقیم تغییر نکرده است.
- گزارش خطای ساختاریافته: همه فازها Diagnostic‌هایی با severity، message، filename، line، column و length تولید می‌کنند.
- ادامه پردازش پس از خطا: Lexer با INVALID و Parser با Panic Mode تلاش می‌کنند بخش‌های سالم بعدی را نیز تحلیل کنند.
- استفاده از Binding معنایی: Navigation و Rename بر اساس Symbol واقعی عمل می‌کنند، نه صرفاً تطبیق متن.

۱.۳ مسئولیت خروجی‌ها

خروجی	تولیدکننده	کاربرد
Token Stream	lexkon.py	ورودی Parser و گزارش مکان دقیق
AST	parskon.py	نمایش ساختار نحوی و ورودی تمام تحلیل‌های بعدی
ANSI / highlighted.html	highlighterkon.py	نمایش رنگی کد در Terminal و Browser

کاربرد	تولیدکننده	خروجی
Scope, Symbol, Type, Diagnostic, Hover و Completion	phase2kon.py	PhaseTwoResult
CFG, Data-Flow, Call Graph, Navigation, Dead Code و Rename	phase3kon.py	PhaseThreeResult
نمایش گرافیکی CFG هر تابع	cfgvisualkon.py	cfg_report.html
نمایش گراف فراخوانی، Recursion و Dead Function	callgraphvisualkon.py	callgraph_report.html

۲. ساختار پوشه‌ها و مسئولیت ماژول‌ها

مسئولیت اصلی	فایل
نقطه شروع و اتصال همه فازها، نمایش خروجی‌ها و اجرای CLI	mainkon.py
توکن‌سازی کاراکتر به کاراکتر، مکان‌یابی و خطاهای لغوی	lexkon.py
Panic Mode و AST، تعریف Nodeهای AST، Recursive-Descent Parser	parskon.py
رنگ‌آمیزی ANSI و تولید HTML	highlighterkon.py
مدل یکپارچه Error، Warning و Info	diagnostickon.py
مدل Symbol، Function Signature و Reference	symbolkon.py
Name Resolution و Symbol، ثبت Scope Tree	scopekon.py
Binding، تعریف/استفاده، Shadowing و خطاهای معنایی	semantickon.py
محاسبه و بررسی Type و Annotation روی AST	typekon.py
Hover و Cursor، Completion، تشخیص Context	intellisensekon.py
هماهنگ‌سازی ماژول‌های فاز دوم	phase2kon.py
ساخت CFG مجزا برای هر تابع	cfgkon.py
Uninitialized Use و Definite Assignment، Liveness	dataflowkon.py
Recursion و Call Graph، Caller/Callee، Reachability، SCC	callgraphkon.py
Find References و Go-to-Definition	navigationkon.py
Editها Safe Rename، Conflict Detection و تولید	refactorkon.py
Dead Store و Dead Function، Unreachable، Unused	deadcodekon.py
هماهنگ‌سازی تمام تحلیل‌های فاز سوم	phase3kon.py
رابط خط فرمان تعاملی و اجرای دستورات	clikon.py

فایل	مسئولیت اصلی
metricskon.py	LOC، تعداد بلوک‌ها، Caller/Callee و Cyclomatic Complexity
cfgvisualkon.py	تولید گزارش HTML/SVG برای CFG
callgraphvisualkon.py	تولید گزارش HTML/SVG برای Call Graph

۲.۱ فایل‌های تولیدی و تست

- README.md: معرفی پروژه، دستورات اجرا، توضیح الگوریتم‌ها، تست‌ها و محدودیت‌ها.
- stress_test.c: ورودی پیچیده برای پوشش Scope، حلقه، Recursion، Unreachable، Dead Function و Data-Flow.
- test*.py: تست‌های مستقل هر ماژول و تست‌های Integration.
- Dockerfile و dockerignore: بسته‌بندی و اجرای پروژه داخل Container.
- compose.yaml: روش اختیاری اجرای تعاملی با Docker Compose.

۳. فاز اول: **Lexer** و قواعد رسمی **Token** ها

۳.۱ روش پیاده‌سازی

Lexer به صورت Hand-Written و کاراکتر به کاراکتر عمل می‌کند. این انتخاب کنترل دقیق روی شماره خط و ستون، Longest Match، Comment، Escape های تودرتو، و Recovery پس از کاراکتر نامعتبر را ممکن می‌کند. در هر موقعیت، اسکنر بر اساس کاراکتر فعلی نوع Token را تشخیص می‌دهد، طول مناسب را مصرف می‌کند و Token شامل نوع، Lexeme و Location می‌سازد.

۳.۲ قواعد اولویت

1. Longest Match یا Maximal Munch: در صورت امکان، طولانی‌ترین Operator یا Token انتخاب می‌شود؛ برای مثال $= >$ یک Token است، نه $=$ و $>$.

2. Keyword قبل از Identifier: ابتدا متن Identifier اسکن می‌شود و سپس با مجموعه Keyword ها مقایسه می‌شود؛ بنابراین while به عنوان KEYWORD ثبت می‌شود.

3. حفظ مکان: Tab، Space و Newline در محاسبه موقعیت لحاظ می‌شوند؛ حتی اگر برای Parser حذف یا نادیده گرفته شوند.

4. بازیابی: کاراکتر ناشناخته به INVALID تبدیل می‌شود و Lexer پس از مصرف آن ادامه می‌دهد.

۳.۳ دسته‌های **Token**

دسته	نمونه	توضیح رسمی
Keyword	int, if, while, return	مجموعه محدود واژه‌های رزرو شده
Identifier	factorial, _count, x1	*[_A-Za-z0-9][_A-Za-z]
Integer	0xFF, 0b1010, 42	ده‌دهی، هگز یا دودویی در دامنه پشتیبانی شده
Float	1e-5, 5., 3.14	بخش اعشاری و Exponent اختیاری
String	"hello\n"	بین Double Quote همراه Escape
Character	'a', '\t'	یک کاراکتر یا Escape بین Single Quote
Operator	==, <-, =>, +	مجموعه Operator های زبان با Longest Match

دسته	نمونه	توضیح رسمی
Delimiter	{ } () ; ,	علائم ساختاری برنامه
Comment	/*...*/ و ...//	تک خطی و Block !Block تودرتو در پیاده سازی پشتیبانی می شود
Whitespace	Space, Tab, Newline	برای Location مهم است و معمولاً وارد Parser نمی شود
Invalid	@ یا متن ناتمام	برای خطای لغوی و ادامه اسکن

۳.۴ قواعد رسمی پیشنهادی برای مستند Lexer

توضیح — این قواعد، زبان مستندشده پروژه را توصیف می‌کنند. جزئیات دقیق مجموعه Keyword و Operator باید با ثابت‌های موجود در lexkon.py یکسان نگه داشته شود.

قواعد Lexical به شکل EBNF/Regex

```
LETTER      ::= [A-Za-z_]
DIGIT       ::= [0-9]
HEXDIGIT    ::= [0-9A-Fa-f]
BINDIGIT    ::= [01]
IDENTIFIER  ::= LETTER (LETTER | DIGIT)*
DEC_INT     ::= 0 | [1-9] DIGIT*
HEX_INT     ::= 0[xX] HEXDIGIT+
BIN_INT     ::= 0[bB] BINDIGIT+
INTEGER     ::= HEX_INT | BIN_INT | DEC_INT
EXPONENT    ::= [eE] [+]? DIGIT+
FLOAT       ::= (DIGIT+ '.' DIGIT* | '.' DIGIT+) EXPONENT?
            | DIGIT+ EXPONENT
ESCAPE      ::= '\\' [nrt0\\"]
STRING      ::= '"' (ESCAPE | [^"\\n]) * '"'
CHAR        ::= "'" (ESCAPE | [^'\\n]) "'"
LINE_COMMENT ::= '//' [^\n]*
BLOCK_COMMENT ::= '/*' ... '*/'
WHITESPACE  ::= [ \t\r\n]+
```

۳.۵ خطاهای لغوی و Recovery

- کاراکتر ناشناخته: Diagnostic با موقعیت دقیق و Token از نوع INVALID؛ سپس ادامه از کاراکتر بعدی.
- String بسته‌نشده: ثبت خطا از محل شروع Quote و جلوگیری از Crash.
- Comment بلوکی بسته‌نشده: ثبت Diagnostic تا انتهای فایل.
- Escape نامعتبر: گزارش مکان و ادامه پردازش در حد امکان.
- EOF: همیشه Token پایان برای توقف امن Parser تولید می‌شود.

نمونه Recovery لغوی

```
int x@ = 5;
int y = 10;
```

خروجی مورد انتظار: @ به‌عنوان INVALID گزارش می‌شود، اما خط بعدی همچنان به‌درستی Tokenize می‌شود.

۴. Parser، گرامر EBNF، AST و Panic Mode

۴.۱ انتخاب الگوریتم

Parser به صورت Hand-Written Recursive Descent پیاده سازی شده است. برای هر Non-Terminal اصلی یک تابع وجود دارد. ساختار Expression با سطوح تقدم و Associativity جدا شده تا Grammar بدون Left Recursion و مناسب LL باشد. این روش برای یک زیرمجموعه آموزشی C خوانا، قابل تست و قابل توسعه است و کنترل مناسبی روی تولید AST و Recovery فراهم می کند.

۴.۲ گرامر EBNF

گرامر پایه زیرمجموعه C

```

program      ::= declaration* EOF ;
declaration  ::= function_decl | var_decl | struct_decl ;
function_decl ::= type_spec IDENT "(" param_list? ")" block ;
param_list   ::= param ("," param)* ;
param        ::= type_spec IDENT ;
type_spec    ::= ("int" | "float" | "char" | "void" | "double") "***" ;
block        ::= "{" statement* "}" ;
statement    ::= if_stmt | while_stmt | for_stmt | return_stmt
               | expr_stmt | block | var_decl ;
if_stmt      ::= "if" "(" expr ")" statement ("else" statement)? ;
while_stmt   ::= "while" "(" expr ")" statement ;
for_stmt     ::= "for" "(" expr_stmt expr_stmt expr? ")" statement ;
return_stmt  ::= "return" expr? ";" ;
expr_stmt    ::= expr? ";" ;
expr         ::= assignment ;
assignment   ::= IDENT ("=" | "+=" | "-=" | "*=") assignment
               | logical_or ;
logical_or   ::= logical_and ("||" logical_and)* ;
logical_and  ::= equality ("&&" equality)* ;
equality     ::= relational ("==" | "!=") relational* ;
relational   ::= additive ("<" | ">" | "<=" | ">=") additive)* ;
additive     ::= multiplicative (("+" | "-") multiplicative)* ;
multiplicative ::= unary (("*" | "/" | "%") unary)* ;
unary        ::= ("-" | "!" | "&" | "**") unary | postfix ;
postfix      ::= primary ("[" expr "]" | "(" arg_list? ")")
               | "." IDENT | "->" IDENT)* ;
primary      ::= INT | FLOAT | STRING | CHAR | IDENT | "(" expr ")" ;
arg_list     ::= expr ("," expr)* ;

```

۴.۳ طراحی AST

AST اطلاعات غیرضروری مانند فاصله‌های سفید را حذف می‌کند و ساختار معنایی برنامه را نگه می‌دارد. هر Node دارای Location است و در فازهای بعدی می‌تواند Scope, Type Annotation, یا Binding دریافت کند.

مفهوم	Node
ریشه برنامه و فهرست Declarationها	Program
نام، Return Type، Parameterها و Body تابع	FuncDecl
Type، نام و Initializer اختیاری	VarDecl
تعریف Struct و Fieldها	StructDecl
فهرست Statementها و Scope بلوکی	Block
ساختارهای کنترل جریان	IfStmt / WhileStmt / ForStmt
دستور بازگشت یا Expression Statement	ReturnStmt / ExprStmt
عملگر، Operandها و تقدم	BinaryExpr / UnaryExpr
نام تابع و Argumentها	CallExpr
برگ‌های AST	Identifier / Literal

۴.۴ Panic-Mode Error Recovery

هنگام خطای نحوی، Parser نباید متوقف شود. روش Panic Mode یک Diagnostic ثبت می‌کند و سپس Tokenها را تا نقطه Synchronization مناسب جلو می‌برد. نقاط معمول شامل {، }، شروع Statement یا EOF هستند. پس از همگام‌سازی، Parser تحلیل بخش‌های سالم بعدی را ادامه می‌دهد.

1. تشخیص Token غیرمنتظره و ثبت پیام شامل Actual، Expected و Location.

2. مصرف Tokenها تا رسیدن به Semicolon، آکولاد بسته یا شروع Declaration/Statement بعدی.

3. ساخت Node ناقص یا عبور امن از بخش خراب؛ جلوگیری از حلقه بی‌نهایت با مصرف حداقل یک Token.

4. ادامه Parse و جمع‌آوری چند خطا در یک اجرا.

معیار ارزیابی مهم — وجود خطا نباید باعث Crash شود. سیستم باید بخش‌های معتبر بعدی را پردازش و خروجی مربوط به آن‌ها را تولید کند.

۴.۵ انتخاب AST به جای CST

AST برای تحلیل معنایی، CFG، Type Checking و Refactoring مناسب‌تر است؛ چون Token‌های تزئینی را حذف می‌کند و مستقیماً مفاهیمی مانند Call و Function، If، Assignment را نشان می‌دهد. Token Stream اصلی همچنان برای Highlighting و گزارش Lexeme دقیق نگه داشته می‌شود.

۵. Syntax Highlighter و زیرساخت مشترک

۵.۱ highlighterkon.py

Highlighter از Token ها و در صورت نیاز اطلاعات AST استفاده می کند تا دسته های مختلف را با Style جدا نمایش دهد. دو خروجی اصلی تولید می شود: ANSI برای Terminal و HTML/CSS برای Browser. خروجی HTML در فایل highlighted.html ذخیره می شود.

- Keyword, Type, Function, Variable, Number, String, Character و Comment رنگ جدا دارند.
- Token های INVALID با Style خطا مشخص می شوند.
- Lexeme اصلی حفظ می شود تا فاصله ها و شکل Source تغییر نکند.
- HTML باید Escape امن برای <, >, & و Quote داشته باشد.

۵.۲ Diagnostic

کاربرد	فیلد
INFO یا ERROR, WARNING	severity
شرح قابل فهم مشکل	message
فایل منبع	filename
محل یکمبنا	line / column
طول بازه مشکل دار	length

DiagnosticBag پیام ها را جمع آوری می کند و امکان فیلتر Errors, Warnings و Infos را فراهم می سازد. یکسان بودن قالب Diagnostic در همه فازها، نمایش و تست خروجی را ساده می کند.

۵.۳ Reference و Symbol

هر Symbol شامل نام، Kind، Type، Signature، محل تعریف، Scope، وضعیت مقداردی و مجموعه Reference است. Reference ها می توانند Read، Write، Call یا انواع دیگر باشند. این داده ها پایه Hover، Navigation، Rename و گزارش Unused هستند.

۵.۴ Name Resolution و Scope Tree

Scope ها به صورت درخت سلسله مراتبی Global، Function، Block و Struct ساخته می شوند. تعریف در Scope محلی ثبت می شود و Resolve از Scope فعلی به سمت والد ادامه پیدا می کند. این روش Shadowing قانونی را از Duplicate Declaration غیرقانونی جدا می کند.

نمونه ساده Scope Tree

```
Global Scope
├── factorial : function (int) -> int
│   └── Function Scope factorial
│       └── n : parameter int
├── main : function () -> int
│   └── Function Scope main
│       └── result : variable int
```

۶. فاز دوم: Type Checking و Semantic Analysis

۶.۱ الگوریتم Semantic Analysis

Semantic Analyzer از روش دوگذری برای Declaration های سطح بالا استفاده می‌کند. در گذر اول، تمام Function ها، Global Variable ها و Struct ها ثبت می‌شوند. در گذر دوم، Body تابع‌ها و_INITIALIZER ها Resolve می‌شوند. این انتخاب باعث پشتیبانی از Recursion و Forward Reference می‌شود؛ زیرا Symbol تابع پیش از تحلیل Body ها در Global Scope موجود است.

1. Reset کردن Scope و Diagnostic های اجرای قبلی.
2. جمع‌آوری تمام Declaration های Global و ساخت Function Signature ها.
3. ساخت Scope جدا برای هر تابع، Block و Struct.
4. Binding هر Identifier و CallExpr به Symbol صحیح.
5. ثبت Read/Write/Call Reference ها و وضعیت استفاده و مقداردهی.
6. گزارش Undefined، Duplicate، Shadowing، Unused و موارد مرتبط.

۶.۲ خطاها و هشدارهای معنایی

- Undefined Symbol یا فراخوانی تابع ناشناخته.
- Duplicate Declaration در یک Scope.
- Shadowing تعریف بیرونی؛ معمولاً Warning.
- Unused Variable یا Unused Parameter.
- استفاده پیش از مقداردهی در سطح تحلیل معنایی یا Data-Flow.
- Recursive Call و Forward Call در صورت وجود Declaration معتبر، خطا نیستند.

۶.۳ Type Checker

Type Checker پس از Semantic Analysis اجرا می‌شود. Type هر Literal، Identifier، Unary/Binary Expression، Call، Assignment و Return محاسبه و در AST Annotation می‌شود. ترتیب رتبه عددی پروژه $\text{char} < \text{int} < \text{float} < \text{double}$ است و تبدیل‌های Narrowing می‌توانند Warning تولید کنند.

نمونه خطا / هشدار	بررسی
تابع int نباید String برگرداند	Return Type

نمونه خطا / هشدار	بررسی
تعداد Argument ها باید با Signature برابر باشد	Argument Count
نوع هر Argument با Parameter سازگار باشد	Argument Type
سمت راست با Type متغیر سازگار باشد	Assignment
عملگر عددی روی Operand مناسب اجرا شود	Operator Operand
Assignment یا مقایسه Pointer در دامنه پشتیبانی شده	Pointer Compatibility
double به int ممکن است کاهش دقت ایجاد کند	Numeric Narrowing

Hover , IntelliSense: Completion .v

Completion v.1

Completion ابتدا موقعیت Cursor را اعتبارسنجی می‌کند، Scope متناظر را پیدا می‌کند و Context را تشخیص می‌دهد. در حالت General، Symbol های قابل مشاهده پیشنهاد می‌شوند. بعد از . یا ->، اعضای Struct در اولویت هستند. در Context آرگومان تابع، Expected Type می‌تواند ترتیب پیشنهادات را بهتر کند.

1. استخراج Prefix از متن قبل Cursor.

2. تشخیص Scope دقیق بر اساس محدوده AST و Cursor.

3. ساخت Candidate ها از visible_symbols یا اعضای Struct.

4. امتیازدهی بر اساس Prefix/Fuzzy Match، فاصله Scope، نوع Symbol و Expected Type.

5. مرتب‌سازی پایدار و محدودکردن تعداد نتایج.

Hover v.2

Hover کلمه زیر Cursor را می‌یابد، آن را در Scope Resolve می‌کند و اطلاعات Symbol را نمایش می‌دهد. خروجی شامل Name، Kind، Type یا Scope، Signature، محل تعریف و تعداد Reference ها است. در صورت وجود Documentation Comment متصل، می‌توان آن را نیز نمایش داد.

ویژگی	مثال خروجی برای factorial
label	factorial
kind	function
detail	int <- (int)
scope	global
definition	main.c:2:5 در نمونه دارای خط خالی اول
references	فراخوانی بازگشتی و فراخوانی داخل main

۷.۳ دلیل استفاده از Symbol Table

Completion و Hover با Symbol Table به Scope و Binding واقعی حساس هستند. در نتیجه دو متغیر هم‌نام در Scope‌های مختلف با هم اشتباه نمی‌شوند و پیشنهادهای فقط از محدوده قابل دسترس ساخته می‌شوند.

موقعیت Cursor — در کد نمونه فعلی که با یک خط خالی آغاز می‌شود و میان دو تابع نیز یک خط خالی دارد، فراخوانی factorial(5) در خط ۹ و نام factorial از ستون ۱۴ شروع می‌شود.

۸. فاز سوم: Control-Flow Graph

۸.۱ ساخت CFG

CFGBuilder برای هر FuncDecl یک ControlFlowGraph مستقل می‌سازد. هر گراف دارای ENTRY و EXIT است و Basic Block های میانی با شناسه B1، B2 و ... ایجاد می‌شوند. Statement های خطی تا زمانی که Branch یا Jump ایجاد نشده در یک Block قرار می‌گیرند.

ترجمه در CFG	ساختار AST
پردازش ترتیبی Statement ها	Block
Block شرط، یال true و false، سپس Merge ضمنی	If
Condition، true به Body، back از Body، false به Condition، به ادامه	While
Init، Condition، Body، Update و Back Edge	For
یال مستقیم به EXIT و پایان مسیر جاری	Return
یال به مقصد خروج Loop	Break
یال به Condition یا Update Loop	Continue
Block غیرقابل دسترسی و قابل گزارش	Statement بعد از Jump

۸.۲ Pending Edge ها

هنگام ساخت بازگشتی CFG، مسیرهای هنوز متصل نشده به صورت PendingEdge نگهداری می‌شوند. وقتی Block بعدی ساخته شد، این Edge ها به مقصد جدید وصل می‌شوند. این روش ساخت If و Loop را بدون نیاز به بازنویسی گراف ساده می‌کند.

۸.۳ DOT و Reachability

- Reachability از ENTRY با پیمایش گراف محاسبه می‌شود.
- Block هایی که از ENTRY قابل دسترسی نیستند با [unreachable] مشخص می‌شوند.
- خروجی Text برای CLI و خروجی DOT برای Graphviz قابل تولید است.

• [cfgvisualkon.py](#) همان اطلاعات را به HTML/SVG تبدیل می‌کند.

شکل مفهومی CFG تابع factorial

```
ENTRY -> B1
B1 [condition: n <= 1]
  true -> B2
  false -> B3
B2 [return 1] -> EXIT
B3 [return n * factorial(n - 1)] -> EXIT
```

Data-Flow Analysis ۸.۴

DataFlowAnalyzer روی CFG هر تابع دو تحلیل تکراری تا نقطه ثابت اجرا می‌کند: Definite Assignment به صورت Basic Block و Forward Must Analysis و Live Variables به صورت Backward May Analysis. برای هر مجموعه‌های LIVE-OUT، DEF، USE، DA-IN، DA-OUT، LIVE-IN و نگاه‌داری می‌شوند.

Definite Assignment

معادلات Definite Assignment

$$\begin{aligned} \text{DA-IN}[B] &= \text{intersection of DA-OUT}[P] \text{ for all reachable predecessors } P \\ \text{DA-OUT}[B] &= \text{DA-IN}[B] \text{ union DEF}[B] \end{aligned}$$

Intersection استفاده می‌شود، چون یک متغیر فقط زمانی حتماً مقداردهی شده است که در تمام مسیرهای ورودی مقداردهی شده باشد. Parameterها و Global Variableها در مجموعه initial_assigned قرار می‌گیرند.

Liveness

معادلات Live Variable

$$\begin{aligned} \text{LIVE-OUT}[B] &= \text{union of LIVE-IN}[S] \text{ for all reachable successors } S \\ \text{LIVE-IN}[B] &= \text{USE}[B] \text{ union } (\text{LIVE-OUT}[B] - \text{DEF}[B]) \end{aligned}$$

Union استفاده می‌شود، چون زنده بودن متغیر در حداقل یک مسیر آینده کافی است. نتایج Liveness برای تشخیص Dead Store به کار می‌روند.

فرایند Fixed Point

1. ساخت DEF و USE هر Block با حفظ ترتیب Eventهای Read و Write.
2. اجرای حلقه تکرار روی Blockهای قابل دسترسی.
3. به روزرسانی مجموعه‌ها تا زمانی که هیچ مجموعه‌ای تغییر نکند.
4. اسکن ترتیب Eventها برای گزارش Use قبل از Definite Assignment.
5. ثبت تعداد Iterationها برای شفافیت و تست.

۹. Dead Code, Call Graph, Navigation, Safe Rename.

۹.۱ Call Graph

هر تابع یک Node و هر CallExpr یک Edge از Caller به Callee است. Callها با استفاده از Binding معنایی Resolve می‌شوند. Queryهای Direct Caller/Callee، Reachable، Reaching و Dead Function روی این گراف اجرا می‌شوند.

برای تشخیص Recursion از Strongly Connected Components استفاده می‌شود. الگوریتم Tarjan در زمان خطی نسبت به تعداد Node و Edge اجرا می‌شود. SCC با بیش از یک تابع نشان‌دهنده Mutual Recursion است و SCC تک‌عضوی با Self Edge نشان‌دهنده Direct Recursion است.

Call Graph کد نمونه

```
factorial -> factorial
main      -> factorial
```

۹.۲ Navigation

- Go-to-Definition: Symbol زیر Cursor را Resolve و محل Declaration را برمی‌گرداند.
- Find All References: تمام Referenceهای همان Symbol را شامل Read، Write، Definition و Call فهرست می‌کند.
- نام‌های برابر در Scopeهای مختلف جدا باقی می‌مانند؛ چون جست‌وجو بر اساس Binding است.

۹.۳ Safe Rename

1. Resolve کردن Symbol انتخاب‌شده و اعتبارسنجی نام جدید به‌عنوان C Identifier.
 2. رد کردن Keyword رزرو شده، Duplicate Declaration و Shadowing خطرناک.
 3. بررسی Reference Capture در Scopeهای داخلی و خارجی.
 4. ساخت Text Edit‌های غیرهم‌پوشان برای Definition و Referenceهای همان Symbol.
 5. اعمال Editها از انتهای Source به ابتدا برای ثابت ماندن Offsetها.
 6. Parse و Semantic Validation مجدد؛ در صورت Conflict، تغییر لغو می‌شود.
- Rename به‌صورت پیش‌فرض Source به‌روزشده را در حافظه برمی‌گرداند. فقط در صورت استفاده از --save فایل خروجی نوشته می‌شود؛ بنابراین فایل اصلی ناخواسته Overwrite نمی‌شود.

Dead-Code Analysis ۹.۴

منبع تحلیل	نوع مشکل
عدم دسترسی از Entry Function در Call Graph	Dead Function
عدم دسترسی Block از ENTRY در CFG	Unreachable Statement
نبود Reference خواندن معتبر در Symbol Table	Unused Variable / Parameter
مقداردهی به متغیری که بعد از آن Live نیست	Dead Store

۱۰. بخش‌های تکمیلی و امتیازی

Code Metrics ۱۰.۱

metricskon.py برای هر تابع اطلاعاتی مانند LOC، تعداد Blockها، Recursive/Dead، Caller/Callee بودن و Cyclomatic Complexity را محاسبه می‌کند. Complexity معمولاً از ساختار CFG و تعداد Branchها به دست می‌آید و برای شناسایی تابع‌های پیچیده استفاده می‌شود.

Metric	کاربرد
LOC	اندازه تقریبی تابع
Basic Blocks	پیچیدگی ساختار جریان کنترل
Cyclomatic Complexity	تعداد مسیرهای مستقل منطقی
Caller / Callee Count	وابستگی و نقش تابع در برنامه
Recursive / Dead	برچسب تحلیل Call Graph

CFG Visualization ۱۰.۲

cfgvisualkon.py گزارش HTML شامل SVG هر تابع تولید می‌کند. ENTRY و Basic Block، EXITها، Edgeهای unreachable و true/false/back Blockهای در خروجی قابل مشاهده هستند. فایل خروجی cfg_report.html است.

Call Graph Visualization ۱۰.۳

callgraphvisualkon.py گراف فراخوانی و جزئیات هر تابع را در HTML/SVG نمایش می‌دهد. Entry، Recursive، Dead و External/Unresolved با Style متفاوت مشخص می‌شوند. فایل خروجی callgraph_report.html است.

۱۰.۴ نکته درباره امتیازی بودن

Docker Deployment در سند پروژه به عنوان Bonus رسمی ذکر شده است. نمایش گرافیکی Call Graph نیز با بخش قابلیت‌های تکمیلی ارتباط مستقیم دارد. CFG Visualization و Code Metrics ارزش افزوده فنی مهمی دارند؛ اما برای ادعای دقیق نوع امتیاز، باید متن نهایی جدول Bonus استاد ملاک قرار گیرد.

۱۱. رابط CLI و راهنمای استفاده

CLI الزام وجود یک رابط قابل اجرا را برآورده می‌کند. کاربر می‌تواند فایل را بارگذاری کند، تحلیل‌ها را اجرا کند و بدون تغییر کد Python به نتایج دسترسی داشته باشد. Prompt اصلی برنامه به شکل `<compoiler>` است.

کاربرد	دستور
نمایش تمام دستورها	help
خلاصه فاز دوم و سوم	summary
نمایش AST، Token، خطاها و Scope Tree	tokens / ast / diagnostics / scope
نمایش Source با شماره خط	source
Completion و Hover در موقعیت یک‌مبنا	complete L C / hover L C
فهرست تابع‌ها و CFG تابع	functions / show-cfg NAME
DEF/USE و DA/Liveness تابع	dataflow NAME
گراف و ارتباط مستقیم تابع‌ها	callgraph / callers / callees
Queryهای Call Graph	reachable / reaching / recursive / scc
گزارش کد مرده	dead-functions / dead-code
رفتن به Definition	goto-def L C
یافتن Referenceها	find-refs NAME / find-refs-at L C
Safe Rename	rename NAME NEW / rename-at L C NEW
نمایش یا ذخیره DOT	callgraph-dot [path]
بارگذاری یا تحلیل مجدد فایل	load PATH / reload
خروج	exit / quit

اجرای مستقیم CLI روی فایل

```
python3 clikon.py program.c
python3 clikon.py program.c --entry main
python3 clikon.py program.c -c "summary" -c "show-cfg main" -c "dead-code"
```

ابهام نام — اگر یک نام در چند Scope وجود داشته باشد، دستور متنی find-refs ممکن است Ambiguous اعلام کند. در این حالت از find-refs-at یا rename-at با line و column استفاده شود.

Docker Deployment ۱۲

Dockerfile از Image سبک Python استفاده می‌کند، مسیر کاری را `app/` قرار می‌دهد، فایل‌های پروژه را `Copy` می‌کند و `mainkon.py` را به‌عنوان `Command` پیش‌فرض اجرا می‌کند. در نتیجه پروژه با یک `docker run` روی سیستم دارای Docker قابل اجرا است.

ساختار Dockerfile

```
FROM python:3.13-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
COPY . /app
CMD ["python", "mainkon.py"]
```

۱۲.۱ ساخت و اجرا

دستورهای اصلی مورد استفاده

```
docker build -t compiler .
docker run --rm -it compiler
```

گزینه `it-` برای CLI تعاملی لازم است. گزینه `rm--` پس از خروج Container را حذف می‌کند. `Image Name` می‌تواند هر نام دلخواهی باشد؛ در اجرای فعلی نام `compiler` استفاده شده است.

۱۲.۲ نگهداری فایل‌های HTML روی میزبان

اجرای Bind Mount در macOS/Linux

```
docker run --rm -it \
-v "$(pwd):/app" \
-w /app \
compiler
```

بدون Volume، فایل‌های `generated` داخل Container موقت ساخته می‌شوند و با `rm--` از بین می‌روند. با Mount کردن پوشه جاری روی `app/`، فایل‌های `highlighted.html`، `cfg_report.html` و `callgraph_report.html` در پوشه پروژه باقی می‌مانند.

۱۲.۳ اجرای تست در Container

نمونه اجرای تست‌ها

```
docker run --rm compiler python testphase3kon.py
docker run --rm compiler python testclikon.py
docker run --rm compiler python testmetricskon.py
docker run --rm compiler python testcfgvisualkon.py
docker run --rm compiler python testcallgraphvisualkon.py
```

مدرک تحویل — از فرمان `docker build`، اجرای `docker run`، نمایش خروجی تحلیل و ظاهرشدن `> compiler` Screenshot تهیه شود. `Dockerfile` و `dockerignore` حتماً در Repository باقی بمانند.

۱۳. راهبرد تست و سناریوهای ارزیابی

ارزیاب ممکن است کد صحیح، خطای لغوی، خطای نحوی یا خطای معنایی بدهد. بنابراین تست باید فقط Happy Path نباشد. معیار اصلی این است که برنامه Diagnostic دقیق تولید کند، Crash نکند و تحلیل بخش‌های معتبر بعدی را ادامه دهد.

۱۳.۱ مراحل تست محیط

1. از داخل پوشه first_build نسخه Python را با `python3 --version` بررسی کنید؛ Python 3.10 یا جدیدتر لازم است.
2. فرمان `python3 mainkon.py` را اجرا کنید و نبود Traceback را بررسی کنید.
3. وجود خروجی Phase Two، Phase Three، ANSI، AST، Token و Prompt برابر `<compoiler` را بررسی کنید.
4. در CLI دستورات `show-cfg`، `main`، `help`، `summary`، `callgraph` را اجرا کنید.
5. با `exit` خارج شوید و فایل‌های HTML را در Browser باز کنید.
6. تست‌های مستقل را اجرا و خروجی Pass را در گزارش یا Screenshot ثبت کنید.
7. همان مراحل را یکبار داخل Docker تکرار کنید.

۱۳.۲ ماتریس تست

نوع تست	ورودی	رفتار مورد انتظار
صحیح	factorial + main	Errors=0، CFG، Recursion، تابع، تشخیص داده شود
لغوی	کاراکتر @ یا String ناتمام	INVALID/Diagnostic و ادامه اسکن
نحوی	Semicolon یا (گم‌شده	Panic Mode و ادامه Parse
معنایی	Duplicate/Undefined/Return mismatch	Diagnosticهای مستقل بدون Crash
Data-Flow	خواندن x بدون Assignment	Potential uninitialized use
CFG	Return سپس Statement	Block/Statement unreachable
Call Graph	تابع استفاده نشده	Dead Function
Rename	نام معتبر و نام دارای Conflict	Apply امن یا Reject با دلیل
CLI	Exit و Unknown command	پیام مناسب و خروج امن

نوع تست	ورودی	رفتار مورد انتظار
Docker	docker run --rm -it compiler	اجرای کامل و Prompt تعاملی

۱۳.۳ تست کامل کد صحیح factorial

کد نمونه دقیق

```
code = """
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

int main() {
    int result;
    result = factorial(5);
    return result;
}
"""
```

شماره خط‌های مهم

مورد	Line	Column
شروع در result در Assignment	9	5
شروع factorial در factorial(5)	9	14
تعریف factorial	2	5
فراخوانی بازگشتی factorial	4	16

علت Line=9 این است که رشته با یک خط خالی شروع می‌شود و بین دو تابع نیز یک خط خالی وجود دارد. چهار Space ابتدای Assignment نیز باعث می‌شود حرف r در ستون ۵ و حرف f در factorial در ستون ۱۴ قرار گیرد.

تنظیمات Cursor

موقعیت‌های پیشنهادی

Completion:	line=9, column=5
Hover:	line=9, column=14

```
Go-to-Definition: line=9, column=14
Find References:   line=9, column=14
Safe Rename:      line=9, column=14
Entry Function:    main
```

خروجی تحلیلی مورد انتظار

خلاصه مورد انتظار برای این ورودی

```
Phase Two Analysis Summary
Errors: 0
Warnings: 0
Infos: 0

Phase Three Analysis Summary
Functions: 2
CFGs: 2
Call edges: 2
Recursive functions: 1
Dead functions: 0
Potential uninitialized uses: 0
```

ممکن است نحوه نمایش جزئیات یا تعداد Dead-code issues با نسخه نهایی Analyzer وابسته باشد، اما نباید خطای غیرمنتظره، تابع مرده یا استفاده مقداردهی نشده در این کد گزارش شود.

دستورهای پیشنهادی CLI

دستورها در Prompt برابر `<compoiler`

```
summary
callgraph
show-cfg main
show-cfg factorial
dataflow main
recursive
goto-def 9 14
find-refs-at 9 14
rename-at 9 14 process
exit
```

۱۳.۴ تست خطا و Error Recovery

الف) خطای لغوی

ورودی

```
int x@ = 5;  
int y = 10;
```

انتظار: Diagnostic برای @ با Location دقیق، تولید INVALID و ادامه Tokenize خط دوم.

ب) خطای نحوی با بخش سالم بعدی

ورودی

```
int broken(int x) {  
    int y = x + 1  
    return y;  
}  
  
int ok() {  
    return 1;  
}
```

انتظار: گزارش Semicolon گم‌شده، Synchronize در ; یا { و ادامه تحلیل تابع ok. برنامه نباید Crash کند.

پ) خطای معنایی و نوع

ورودی

```
int bad(int n) {  
    int x;  
    int x;  
    return "hello";  
}  
  
int ok() {  
    return 1;  
}
```

انتظار: Duplicate declaration برای x و Return type mismatch برای bad؛ تابع ok همچنان تحلیل شود.

Undefined Symbol (ت)

ورودی

```
int main() {  
    return missing;  
}
```

انتظار: Undefined symbol با نام missing و موقعیت دقیق.

ث) استفاده قبل از مقداردهی

ورودی

```
int main() {  
    int x;  
    return x;  
}
```

انتظار: Potential uninitialized use برای x در Data-Flow.

ج) کد غیرقابل دسترسی و تابع مرده

ورودی

```
int unused() { return 42; }  
int main() {  
    return 0;  
    int x = 1;  
}
```

انتظار: unused به عنوان Dead Function و Declaration متغیر x پس از return به عنوان Unreachable گزارش شود.